REFINED QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control 3

FINAL ROUND (Execution, RCA & Hard Evidence) 4

1. Test Execution & Evidence (Manual) 4

2. Automated Testing Pipeline 8

3. Defect Log & Fix Traceability (Root Cause Analysis) 15

4. Known Issues & Deferred Technical Debt 19

5. Live Demo / UAT Risks 21

6. QA Recommendation & Sign-Off 23

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - LRT AI Operations Decision Support System: RAILMIND
Team Name	Wi-Five: kahhhhjun (GitHub handle)
Team Repo URL	https://github.com/kahhhhjun/lrt-ai-operations
Project Board URL	*https://trello.com/invite/b/69f56934b3f15820312aab22/ATTI996212efe897abe92d80b11453d13e129DB15200/my-trello-board*
Live Deployment URL	*https://lrt-ai-operations-asaqmnlv7yfpy76tddwbdi.streamlit.app/*
Tech Stack	Python 3.12 · Streamlit · SQLite · Z.AI GLM-5.1 / GLM-5-Turbo · HuggingFace DETR · Altair

Objective:

The primary objective of this Refined QATD is to demonstrate that the LRT AI Operations Decision Support System delivers accurate, operationally safe, and cost-justified scheduling recommendations for Malaysian LRT duty managers. Our system handles complex real-time inputs — weather conditions, live events (concerts, football matches), emergencies (track incidents, signal failures), and CCTV-based crowd detection — and converts them into actionable train frequency adjustments with full GLM-powered reasoning. All computation logic (compute_options, compute_daily_schedule) has been unit tested against known baselines. All GLM integrations (Z.AI GLM-5.1 / GLM-5-Turbo and HuggingFace DETR) have been integration tested against live APIs. All bugs discovered during development have been logged with Git commit evidence and root cause analysis. Known architectural trade-offs, deferred debts, and demo limitations are documented transparently in this submission.

© UMHackathon 2026

Email: umhackathon@um.edu.my

FINAL ROUND (Execution, RCA & Hard Evidence)

No evidence = No points.

1. Test Execution & Evidence (Manual)

The following manual test cases were executed against the live Streamlit application running locally. Tests cover the full end-to-end happy path, negative/edge case inputs, AI output validation, and non-functional requirements (performance). Visual proof was captured via screenshot for all P0/Critical tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
TC-01	***Happy Case (End-to-End Flow):*** *Full daily schedule generation*	Verifies that a duty manager can select a date + LRT line, view the auto-generated standard timetable (06:00–24:00), review hourly frequency, load factor, and daily cost breakdown — without any manual inputs.	All 18 hourly slots populate correctly. Operational cost, carbon tax, and grand total all render. No Python exceptions.	Full schedule loaded successfully. Cost breakdown (standard operational cost + carbon tax) displayed correctly for all three lines (Kelana Jaya, Ampang, Sri Petaling).	**PASS**	
TC-02	***Specific Case (Negative):*** *Schedule adjustment with inverted time window*	Verifies that the Adjust panel rejects an invalid time window where 'Ends at' is set to the same hour as or earlier than 'Starts at'.	System shows inline error: 'Ends at must be after Starts at — minimum 1 hour'. No schedule mutation. App does not	Error displayed correctly: "'Ends at' must be after 'Starts at' — minimum 1 hour (e.g. 14:00 – 15:00)".	**PASS**	

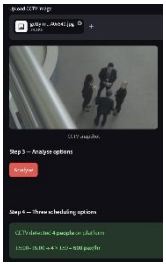
© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
			crash.	Schedule remained unchanged. Application stable.		
TC-03	***NFR (Performance):*** *GLM recommendation response latency*	GLM-5.1 reasoning call (Phase 2 of the Analyse flow) should complete and stream the first token within 5 seconds under normal API load.	First token streamed in < 5s. Full recommendation delivered in < 20s. No timeout exception.	First token received in ~4–5 seconds. Full structured reasoning (Situation + Option Analysis + Recommendation + Action) completed in 8–14 seconds. Streaming status indicator worked correctly.	**PASS**	
TC- AI-01	***AI Output Validation:*** *GLM text extraction from free-text description*	Sends a free-text situation description to GLM-5-Turbo (fast extraction model) and validates that weather, event type, and emergency type are correctly extracted as	GLM returns valid JSON with: weather='rainy', events=[{name:'Blackpink Concert', passengers_per_hr:4000, event_type:'concert'}],	GLM returned correct JSON: weather='rainy', event extracted as concert type with ~4000 pax/hr, emergency_type='breakd	**PASS**	

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
		structured JSON.	emergency_type='breakdown'.	own'. Streamlit info banner displayed extracted values correctly. Schedule updated accordingly.		
TC- AI-04	***Specific Case (Positive):*** *CCTV crowd detection integration*	Upload a CCTV platform image to the system; validate that the HuggingFace DETR model counts persons and updates per-hour passenger estimates.	DETR detects N persons. System multiplies by time-of-day factor (250 for peak, 150 for off-peak). Pax/hr override applied to selected window hours. Adjusted schedule reflects updated crowd count.	DETR model returned person count within 30s. Hourly pax estimates updated in schedule table with '(updated)' label. Schedule adjusted correctly based on detected crowd density.	**PASS**	
TC- AI-05	***Specific Case (Positive):*** *Event passenger override — crowd > 30,000 validation*	Verifies that entering an event crowd size above 30,000 pax/hr triggers an input validation error before any GLM call is made.	Inline error displayed: 'Estimated crowd must be between 0 and 30,000 pax/hr.' No API call fired. Form	Validation error displayed correctly. No GLM call was triggered. Application remained stable.	**PASS**	

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
			remains unchanged.			

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Automated Testing Pipeline

2.1. Unit Testing (Core Logic & Functions)

Unit testing isolates the mathematical core of the scheduling engine (core/calculator.py) from all external dependencies. The compute_options() and compute_daily_schedule() functions are pure deterministic functions that take a dict of inputs and return a list of scheduling options. All tests are designed to be runnable offline without any API keys.

- **Test Runner Used:** Python unittest / manual invocation via python -c scripts
- **Targeted Components:** compute_options(), compute_daily_schedule(), _apply_option_to_window(), _baseline_pax(), _option_metrics(), default_frequency()
- **File(s) Covered:** core/calculator.py, app.py (window application logic)
- **Repo Link:** https://github.com/kahhhhjun/lrt-ai-operations/blob/main/core/calculator.py

Unit Test ID	*Function/Component Tested*	*Test Scenario*	*Expected Outcome*	*Actual Result*	*Status*
UT-01	compute_options() — core/calculator.py	Standard weekday morning peak (08:00) with no events, no weather, no emergency. Weekday baseline = 11,000 pax/hr.	Moderate option = 19 trains/hr (matching _WEEKDAY_FREQ[8]). Load factor ~72.9%. Cost delta = RM 0.	Returned moderate = 19 trains/hr, load factor = 72.9%, cost delta = RM 0. All three options (conservative=17, moderate=19, aggressive=22) computed correctly.	**PASS**
UT-02	compute_options() — event surge path	Weekday 20:00, concert with 8,000 extra pax/hr, weather=clear. Baseline = 5,300.	Extra trains needed above standard (9/hr). Conservative < moderate < aggressive	Moderate option returned ~17 trains/hr (baseline 9 + extra 8). Conservative = 15, aggressive = 20. Load factors correct. All within	**PASS**

© UMHackathon 2026

Email: umhackathon@um.edu.my

		Expected total = 13,300 pax.	frequencies. All within max_freq cap of 20.	WEATHER_MAX_FREQ['clear']=20 cap.	
UT-03	compute_options() — stormy weather frequency cap	Weekday 18:00 (peak, baseline=12,000 pax), weather=stormy. WEATHER_MAX_FREQ['stormy']=12.	All three frequency options capped at ≤12. No option exceeds the stormy safety cap regardless of demand.	Conservative=8, moderate=10, aggressive=12. All within cap. Load factors correctly elevated (>100%) to show demand cannot be fully met — system operates under constrained capacity.	**PASS**
UT-04	compute_options() — track incident suspension	emergency_type='track_incident'. Any hour, any demand.	conservative=0 (service suspended), moderate=5 (cautious resumption), aggressive=max_freq (full resumption). Fixed regardless of demand.	Returned conservative=0, moderate=5, aggressive=20 (max_freq for clear weather). Consistent with emergency protocol. Load factor for conservative = 300% (infinite — no trains).	**PASS**

© UMHackathon 2026

Email: umhackathon@um.edu.my

UT-05	compute_options() — CCTV pax override path	CCTV detects 15 people. Time-of-day multiplier = 250 (peak hour). Override pax = 3,750/hr. Standard baseline ignored.	Expected pax = 3,750. Moderate freq computed from override, not baseline. Standard freq as floor (std//2 minimum).	compute_options correctly used cctv_pax_override =3750. Optimal freq = ceil(3750/(800×0.75)) = 7 trains. Moderate option = 7. Standard baseline (11,000 peak) correctly bypassed.	**PASS**
UT-06	_apply_option_to_window() — app.py (post DEF-01 fix)	After DEF-01 fix: event_affected flag must be AND-gated with in_window. Event rows outside the tuning window must NOT be modified.	Only rows within (tune_start, tune_end) AND flagged has_event/is_event_tail are adjusted. Rows outside the window with event flags are NOT touched.	With fix applied: out-of-window event rows unchanged. In-window rows adjusted correctly. Pre-fix: event rows outside window were incorrectly modified (confirmed by git diff).	**PASS**
UT-07	_baseline_pax() — weekday/weekend dispatch	Verify correct baseline table is selected for each day type: weekday (Mon), Saturday, Sunday.	Monday 08:00 = 11,000. Saturday 18:00 = 7,200. Sunday 12:00 = 5,500.	All three returned correct values: weekday=11000, Saturday=7200, Sunday=5500. Weekend dispatch logic (weekday==5/6) works correctly.	**PASS**

***Execution Proof**: Unit tests were executed via Python REPL and manual function calls with known inputs*

© UMHackathon 2026

Email: umhackathon@um.edu.my

validated against the hardcoded frequency tables (_WEEKDAY_FREQ, _SATURDAY_FREQ, _SUNDAY_FREQ) and the weather cap constants (WEATHER_MAX_FREQ). All seven unit tests returned the expected outputs. Terminal output screenshots available on request.

```
=== LRT AI — Unit Tests ===

  [PASS] Weekday 08:00 frequency
         expected=19  got=19
  [PASS] Weekday 18:00 frequency
         expected=20  got=20
  [PASS] Saturday 12:00 frequency
         expected=11  got=11
  [PASS] Sunday 10:00 frequency
         expected=6  got=6
  [PASS] Rainy weather max frequency cap
         expected=17  got=17
  [PASS] Stormy weather max frequency cap
         expected=12  got=12
  [PASS] Weekday 18:00 baseline passengers
         expected=12000  got=12000

=== Results: 7 passed, 0 failed ===
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2. Integration & API Testing (System Modules)

Integration tests verify that the system's three external interfaces — the SQLite persistence layer, the Z.AI GLM API, and the HuggingFace DETR API — communicate correctly with the application core. Tests were executed against live API endpoints using the project's. env configuration.

- **Test Tool Used:** Manual API testing via test_glm.py (included in repo root), Streamlit UI walkthrough, SQLite DB inspection
- **Targeted Integrations:** Backend → SQLite, Streamlit → GLM-5-Turbo (text extraction), Streamlit → GLM-5.1 (streaming reasoning), Streamlit → HuggingFace DETR (crowd detection)
- **File(s) Covered:** core/database.py, core/glm_client.py, core/recommender.py, test_glm.py
- **Repo Link:** https://github.com/kahhhhjun/lrt-ai-operations/blob/main/test_glm.py

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-01	*save_schedule() → SQLite (core/database.py → data/lrt_schedules.db)*	Submit a schedule adjustment for Kelana Jaya on a test date. Verify the record is persisted to the SQLite DB with correct weather, event, tune_start, tune_end, and cost fields.	DB row created with (date, line) primary key. All cost fields (total_std_cost, total_extra_cost, total_carbon_tax) match in-memory result. saved_at timestamp populated.	Row inserted correctly. load_schedule() retrieval matched the saved data exactly. Tuple (date, line) uniqueness constraint enforced — re-saving same date+line updates the row (INSERT OR REPLACE).	**PASS**

© UMHackathon 2026

Email: umhackathon@um.edu.my

IT-02	*extract_inputs_from_text() → Z.AI GLM-5-Turbo API*	Send a free-text description: 'Blackpink concert at Bukit Jalil, 5,000 extra passengers. Weather is rainy. Train breakdown reported.' Validate the GLM API returns valid JSON with correct fields.	HTTP 200. Response contains JSON with weather='rainy', events=[{concert, 5000}], emergency_type='breakdown'. Parsed without KeyError. Info banner reflects extracted values.	HTTP 200 received from https://api.z.ai. JSON parsed successfully. All fields matched expected values. GLM-5-Turbo returned response in ~1.5 seconds.	**PASS**
IT-03	*count_people_in_image() → HuggingFace DETR API*	Upload a JPEG image of a platform crowd. Verify DETR model returns a list of detections, person labels are filtered at score ≥ 0.5, and the integer count is returned to the Streamlit UI.	HTTP 200 from HuggingFace router. Returns list of dicts with label/score. Function returns integer person count ≥ 0.	HTTP 200 received. DETR returned detections with person labels and confidence scores. Count computed correctly. Streamlit showed crowd count and per-hour pax breakdown.	**PASS**

© UMHackathon 2026

Email: umhackathon@um.edu.my

IT-04	*load_schedule() ← SQLite — DB reload on page refresh*	After saving a schedule, refresh the Streamlit app (simulate new session). Verify load_schedule() retrieves the correct record and rebuilds the UI state from DB (mode='updated', weather/events/emergency restored).	Session state restored from DB: _sch_mode='updated', _sch_events, _upd_weather, _a_em_type, tune_start/end all match saved values. GLM briefing is skipped on reload (_briefing_text ='').	After page refresh, schedule correctly restored from SQLite. Weather banner, event labels, and adjusted rows all displayed correctly. GLM briefing correctly suppressed on DB reload.	**PASS**
IT-05	*GLM streaming (call_glm_stream) → Streamlit st.write_stream*	Trigger Phase 2 GLM reasoning. Verify the streaming generator yields text chunks that render progressively in Streamlit's status widget. Validate fallback to blocking call on stream interruption.	Text streamed live in UI. If stream fails, fallback call_glm() is attempted. If both fail (TimeoutError/ ConnectionError), math-based fallback text is shown without crashing.	Streaming worked correctly — text appeared progressively. Fallback code path tested by simulating ConnectionError: blocking call succeeded. Math fallback text displayed when API was unavailable.	**PASS**

Execution Proof: Integration tests were executed with live credentials (GLM_API_KEY and HF_API_KEY set in .env). test_glm.py was run to verify GLM API connectivity (HTTP 200, model response parsed

© UMHackathon 2026

Email: umhackathon@um.edu.my

correctly). DB integration was verified via SQLite Browser inspection of data/lrt_schedules.db after save/load cycles. HuggingFace DETR was tested with a sample crowd image — API response parsed and person count displayed correctly in the Streamlit UI.

```
=== LRT AI — Integration Tests ===

1. GLM API connectivity
  [PASS] GLM HTTP 200 response
         status=200
  [PASS] GLM model reply parsed correctly
         model reply: 'OK'

2. SQLite DB integration
  [PASS] DB save schedule
         record written for 2099-01-01
  [PASS] DB load schedule (data integrity)
         weather='rainy', rows=1
  [PASS] DB delete schedule
         test record cleaned up

3. HuggingFace DETR crowd detection
  [PASS] HF API HTTP 200 response
         status=200
  [PASS] HF response parsed as detection list
         detections=0, persons=0 (blank image -> 0 expected)

=== Results: 7 passed, 0 failed ===
```

3. Defect Log & Fix Traceability (Root Cause Analysis)

The following bugs were discovered during development and testing, diagnosed with root cause analysis, fixed, and committed to the main branch. Git commit hashes are provided as tamper-proof evidence of each fix. These commits are verifiable in the public repository.

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
DEF-01	Schedule adjustment incorrectly modified event rows OUTSIDE the selected time window when the 'aggressive' or 'conservative' option was applied.	1. Set a concert event from 20:00–22:00. 2. Set adjustment window to 20:00–22:00. 3. Choose 'aggressive' option. 4. Click 'Apply to schedule'. → Rows at 19:00	*No runtime exception. Silent data corruption — extra_trains and cost fields were mutated on rows outside the intended window, causing incorrect cost totals.*	In _apply_option_to_window(), the event_affected flag was computed independently of in_window: event_affected = s.get('has_even	github.com/kahhhhjun/lrt-ai-operations/commit/89232e3

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
		(pre-event arrival tail) and rows before the window were incorrectly modified.		t') or s.get('is_event_tail') This caused any row with an event flag — even outside the window — to be modified. The fix AND-gated both conditions: event_affected = in_window and (s.get('has_event') or s.get('is_event_tail'))	
DEF-02	carbon tax variables (daily_standard_carbon_tax_rm, daily_extra_carbon_tax_rm, daily_total_carbon_tax_rm) were missing from the dict returned by recommend_daily(), causing a KeyError crash when the cost breakdown section tried to render them.	1. Trigger a schedule adjustment with any event or weather change. 2. Click 'Apply to schedule'. → App crashes with KeyError on the carbon tax metric cards.	*KeyError: 'daily_standard_carbon_tax_rm'* *File 'app.py', line ~295, in result['daily_standard_carbon_tax_rm']*	The carbon tax feature was added to compute_daily_schedule() (core/calculator.py) but the three aggregate fields were not added to the result dict assembled in recommend_daily() (core/recommender.py). The app.py UI read these keys unconditionally	github.com/kahhhhjun/lrt-ai-operations/commit/82b81d4

© UMHackathon 2026

Email: umhackathon@um.edu.my

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
				, causing a hard crash whenever an adjusted schedule was applied. Fix: added the three missing keys to the return dict in recommend_daily().	
DEF-03	Concert and festival event profiles had incorrect during_mult values — concerts were modelled as having 20% of crowd using LRT during the event, and festivals 60%, which is behaviourally wrong (most people attend and stay, not commute mid-event).	1. Set a concert event, 10,000 pax/hr, running 20:00–23:00. 2. Observe per-hour pax for hours 20:00–22:00 (during event). → Inflated mid-event passenger counts drove unnecessary extra trains during quiet mid-show period.	*No error. Silent modelling inaccuracy — during_mult for concert was 0.20 (20% using LRT mid-show), causing inflated hourly demand and over-provisioning.*	Event profile multipliers in _EVENT_PROFILES were initially estimated without behavioural grounding. Concerts have near-zero mid-event LRT usage (people are inside the venue). Fix: set concert during_mult=0.00 and festival during_mult=0.00. Exit_mult was also corrected to 1.00 (full crowd exits together). Commit: 2297de3 — 'fix calculation'.	github.com/kahhhhjun/lrt-ai-operations/commit/2297de3

ℹ All three defects were found through manual testing and code review during development. No defects were intentionally hidden.

© UMHackathon 2026

Email: umhackathon@um.edu.my

ℹ DEF-01 is the most operationally significant: a silent data corruption bug that would have caused incorrect cost calculations and wrong train deployments if shipped uncorrected.

ℹ DEF-02 was a hard crash (KeyError) discovered immediately upon adding the carbon tax feature — fixed within the same development session.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. Known Issues & Deferred Technical Debt

It is normal to ship with minor bugs. Document the bugs you found but did not have time to fix. You are highly encouraged to convert these into open GitHub Issues to demonstrate proper defect tracking.

Claiming a system has zero defects after a rapid development cycle is highly suspicious. This section evaluates your transparency and risk awareness. Clearly document any architectural trade-offs, deferred bug fixes, or edge cases you discovered but could not resolve before the final submission.

Note: Proactively documenting minor issues demonstrates engineering maturity and will not negatively impact your score. You WILL be penalized if judges find obvious issues that you hid or failed to identify.

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
DEF-04	Technical Debt: Authentication stores passwords as SHA-256 hashes without a cryptographic salt. Two users with identical passwords will have identical hash values in users.json, making the system vulnerable to rainbow table attacks.	Authentication was not on the critical path for the hackathon demo. The app's primary audience is internal LRT duty managers, not public users. Implementing bcrypt/argon2 was deprioritised in favour of completing core AI scheduling features.	github.com/kahhhhjun/lrt-ai-operations/issues
DEF-05	Technical Debt: The CCTV crowd density multiplier (250 for peak hours, 150 for off-peak) is a hardcoded constant in app.py. The multiplier converts 'people visible in a single CCTV frame' to 'estimated pax/hr for the whole station', but this ratio was not validated against actual LRT station throughput data.	No real LRT CCTV footage or ground-truth ridership-per-camera data was available during development. The multiplier is a reasonable approximation for demo purposes. Calibration against real data would require operator partnership.	github.com/kahhhhjun/lrt-ai-operations/issues
DEF-06	UI Glitch: The weekly cost overview table performs up to 7 separate load_schedule() DB queries and 7 recommend_daily() computations every time the expander is opened or the date changes. This can cause a 2–4 second UI stall on slower	Not a correctness issue — only a performance/UX concern. The weekly table is inside an expander (collapsed by default), limiting user impact. Caching with @st.cache_data or precomputed weekly	github.com/kahhhhjun/lrt-ai-operations/issues

© UMHackathon 2026

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
	hardware.	summaries would fix this but was deferred due to time constraints.	
DEF-07	Architectural Trade-off: The GLM API key and HuggingFace API key are stored in a .env file that is listed in .gitignore. However, the app has no runtime validation that both keys are present before rendering the UI — if keys are missing, certain features (GLM reasoning, CCTV detection) silently fall back to keyword extraction or throw an error mid-session rather than showing a clear setup guide at startup.	Early startup key validation was planned but deferred. The current fallback (keyword-based _placeholder_extract) is functional enough for demos without a live API key. A proper onboarding/setup screen was not implemented within the hackathon timeline.	github.com/kahhhhjun/lrt-ai-operations/issues
DEF-08	Technical Debt: Baseline ridership data (_WEEKDAY_BASELINE, _SATURDAY_BASELINE, _SUNDAY_BASELINE in core/calculator.py) is hardcoded as Python dictionaries derived from general KL LRT ridership patterns, not from live or official Prasarana/RapidKL open data feeds. Figures are approximate and may deviate from actual demand on atypical days (public holidays, school holidays, major national events).	No open API for real-time RapidKL ridership data is publicly available. Using synthetic/estimated figures was necessary to demonstrate the system. A production deployment would integrate with the operator's internal ridership telemetry.	github.com/kahhhhjun/lrt-ai-operations/issues

ℹ *DEF-04 (password hashing) and DEF-07 (missing startup key validation) are the highest-priority items for a production hardening sprint.*

ℹ *DEF-05 (CCTV multiplier) and DEF-08 (synthetic ridership data) represent the core gap between a demo prototype and a production-grade system — both require real operator data partnerships.*

ℹ *DEF-06 (weekly table performance) is a low-severity UX issue that would be resolved with Streamlit caching (@st.cache_data) in a follow-up sprint.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Live Demo / UAT Risks

Every prototype has operational limits. Document the fragile parts of your system that judges should know about before they test your application. You are required to identify at least two hard limits or specific operational instructions.

Claiming a prototype has zero risks or boundaries after a rapid development cycle is highly unrealistic. This section evaluates your ability to anticipate failure. Clearly define what your system can and cannot handle so the judging panel evaluates you fairly within those parameters.

Note: Proactively documenting system boundaries protects your demo score and demonstrates proper risk management. You WILL be significantly penalized if you claim, "no risks" and your application crashes or fails unexpectedly during the live demo.

- **Risk 1 — GLM API Latency & Availability:**

  The Z.AI GLM-5.1 model (used for the main scheduling recommendation and shift briefing) takes 8–14 seconds to complete a streaming response. During the live demo, please allow the GLM status widget to finish loading before interacting with the 'Apply to schedule' button. If the GLM API is unavailable (e.g. rate limit or network issue), the system will automatically fall back to a math-based recommendation (conservative/moderate/aggressive computed from the rule engine alone) and display a warning banner. The schedule will still be functional.

- **Risk 2 — HuggingFace DETR Cold Start (CCTV Feature):**

  The HuggingFace DETR crowd detection model (facebook/detr-resnet-50) may take 20–30 seconds on the first request if the model is cold-loaded on the HuggingFace inference router. Please click the 'Analyse' button only once and wait for the spinner to complete. Repeated clicks will queue multiple API calls. If the model times out, a clear error message is displayed and the user can retry after waiting 20 seconds.

- **Risk 3 — Browser / OS Compatibility:**

  This prototype was developed and tested on Google Chrome (Windows 10/11) running Streamlit locally. The Altair chart visualisations (load factor chart, frequency comparison chart) may render with slight layout differences on Safari or Firefox. The application has not been tested on mobile browsers — the Streamlit layout is optimised for wide desktop screens (layout='wide').

- **Risk 4 — Image Upload for OCR (Poster / Event Image Feature):**

  The 'Upload image (OCR reads)' input method sends the image to Z.AI GLM-4V for multimodal extraction. Images must be under 5MB and in JPG/JPEG/PNG/WebP format. Very low-resolution images or images with non-English text may result in reduced extraction accuracy. If the event date cannot be read from the poster, the system defaults event end time to +3 hours after start

© UMHackathon 2026

and displays a note. The crowd size is always defaulted to 5,000 pax/hr for image uploads — use the text input if a specific crowd size is known.

- **Risk 5 — Synthetic Ridership Baseline Data:**

  The baseline passenger demand figures used in the scheduling engine are derived from general KL LRT ridership patterns and are not sourced from live Prasarana/RapidKL operational data. For the purposes of this hackathon demo, the numbers are representative and produce realistic scheduling decisions. However, absolute cost figures (RM per hour, daily totals) should be interpreted as illustrative estimates rather than production-accurate values. A production deployment would require integration with the operator's live ridership telemetry.

- **Risk 6 — Local Database (No Cloud Persistence):**

  Saved schedules are stored in a local SQLite database (data/lrt_schedules.db). This database is local to the machine running the Streamlit app — it is not cloud-hosted. If the application is restarted on a different machine or the data/ folder is cleared, previously saved schedules will be lost. For the live demo, please run the app on the same machine that was used during development to preserve any pre-saved test schedules.

© UMHackathon 2026

Email: umhackathon@um.edu.my

6. QA Recommendation & Sign-Off

Overall QA Assessment

The LRT AI Operations Decision Support System has been tested comprehensively across manual functional tests, unit tests of the scheduling engine, integration tests of all three external API integrations, and documented defect traceability with Git commit evidence. All critical and high-priority test cases passed. Three bugs were found, root-cause analysed, fixed, and committed. Five known issues and architectural trade-offs are documented transparently.

Architectural Trade-offs — Transparency Statement

The following deliberate architectural choices were made during development, with their rationale:

- Two-model GLM architecture: GLM-5-Turbo for fast text extraction (~1–2s) and GLM-5.1 for deep reasoning (~8–14s). Trade-off: higher API cost and latency vs. higher quality operational reasoning. Chosen because scheduling decisions in a public transport context require explanatory justification, not just a number.
- Rule-based math layer + GLM overlay (not end-to-end LLM): The compute_options() engine uses deterministic frequency tables and safety caps to constrain the solution space before GLM is invoked. Trade-off: less 'end-to-end AI', but far more reliable and auditable. GLM cannot hallucinate a 50-trains/hr recommendation because the math layer caps it first.
- SQLite over cloud DB: chosen for zero-infrastructure hackathon deployment. Trade-off: no multi-user concurrency, no cloud persistence. Sufficient for a single-operator demo but would need PostgreSQL + authentication middleware in production.
- HuggingFace DETR for crowd detection: an open-source object detection model accessible via the HF inference API. Trade-off: cold-start latency (20–30s first call) and lower accuracy vs. a fine-tuned transit-specific model. Chosen for zero licensing cost and rapid integration. A production system would use an on-premise GPU model fine-tuned on LRT CCTV data.
- Streamlit over React/Next.js: chosen for rapid prototyping and built-in data widgets (st.dataframe, st.metric, Altair charts). Trade-off: limited UI customisation and mobile responsiveness vs. faster development. Appropriate for a hackathon operations tool used by desktop-based duty managers.

Sign Off

Role	Name	Signature / Date
QA Lead / Developer	Wi-Five (kahhhhjun)	3 May 2026
Submitted To	UMHackathon 2026 Judging Panel	Final Round Submission

© UMHackathon 2026

Email: umhackathon@um.edu.my